

Model submissions — demonstration paper (Kueh et al., BMJ Open 2026)

Worked, **Exceptional-level** example submissions for every project increment and milestone, based on the course's demonstration paper (MASLD $\times$ BMI/WHtR $\times$ mortality, NHANES 2007–2018). Each assignment has its **own subfolder** containing:

- the **submission** (what a strong student group hands in), and
- a step-by-step **.qmd** that produces the numbers in that submission; the code is the source of truth and the prose follows it (plus a rendered **.html** so students can read code + output without running it).

These accompany the assignment handouts in `../practice/` — `P1_analytic_cohort.md` ... `P5_missing_data.md`, and `README.md`. Students do this on **their own** M0-locked paper; this is the instructor's answer key.

Two kinds of example, and why the difference matters

Every file here is an **Instructor exemplar**: technically correct as far as the analyses establish. None contains a planted error.

That matters for M2. Your critique target is another *group's* M1, not one of these — and a good M1 still has real things to criticise. In this M1 they are: an adjustment set whose causal roles are unjustified, a dominant-threat ranking that puts an observed-but-ambiguous problem above an untested one, and an interaction that is design-sensitive and unresolved. Criticising open questions in sound work is the normal case in peer review, and it is what M2 asks you to do.

If a file is ever added as a deliberately flawed **Review case**, it will say so in its first line and its folder will be named ***_review_case**. Nothing here is one today.

Contents

Assignment	Subfolder	Submission	Reproducible code	Reproduces
P1 Analytic cohort	<code>P1_analytic_cohort/</code>	<code>submission.md</code>	<code>P1_code.qmd</code>	typed funnel; locked domain N = 6,048
P2 Confounding / Table 1	<code>P2_confounding/</code>	<code>_submission.md</code>	<code>P2_code.qmd</code>	Table 1; PIR swap; replication IV vs I 16.05->3.79, locked IV vs III 2.34->1.06, matched rows
P3 Effect modification	<code>P3_effect_modification/</code>	<code>submission.md</code>	<code>P3_code.qmd</code>	targeted 1-df IV vs III x sex: ratio of HRs 1.00 (0.42-2.35); 6-yr risk differences
P4 Design-aware estimate	<code>P4_design_aware/</code>	<code>_submission.md</code>	<code>P4_code.qmd</code>	design built on the frame then <code>subset()</code> ; adjustment x design matrix on matched rows

Assignment	Subfolder	Submission	Reproducible code	Reproduces
P5 Missing data	P5_missing_data/	_submission.md	P5_code.qmd	Model-1 missingness 1.8% (cancer only); complete-case 2.97, stress 2.98/2.99, MI 2.98
M1 Replication deck	M1_replication/	_deck.md	M1_code.qmd	all five increments + figures + dominant-threat judgment
M2 Critique	M2_critique/	_critique_deck.md	NOTE_reproducibility.qmd	critique of the M1 deck above — no new numbers)
M3 Oral defense	M3_defense/	_defense.md	M3_defense_evidence.qmd	the numbers behind each defense answer
M4 Reanalysis letter	M4_reanalysis_letter/	letter.md	M4_code.qmd	every claim in the letter

Running the code

Each .qmd reads the built cohort from the reproduction project:

```
source("../_shared/paths.R")
repro <- find_repro()      # walks up for example/repro or reproduction
```

So ../reproduction/data/derived/masld_analytic.rds and merged_all.rds have to exist before an increment will run. **They are not in this repository** — the NHANES microdata behind them is large and freely re-downloadable — so build them once by running ../reproduction/R/run_all.R, which fetches what it needs from CDC on its first run. After that, from a subfolder:

```
quarto render P1_code.qmd      # writes P1_code.html
```

Requires R (≥ 4.4) with dplyr, survival, survey, mice, rms, jsonlite and Quarto (≥ 1.4). All code files render cleanly and reproduce the numbers cited in the matching submission.

You do not have to run anything to read these examples. Every .qmd has a rendered .html beside it and every submission has a rendered .pdf. Running the code is for when you want to change something and see what moves.

Order matters: M1 assembles, it does not re-implement

Each increment ends with an emit chunk that writes a result object to _results/. Each object holds a flat set of **facts** — named, already-formatted values — alongside the fuller tables. Those files are in the repository, so you can read them without running anything:

```
_results/P1.json    funnel, group sizes, deaths, alcohol codings
_results/P2.json    the two-estimand attenuation table
_results/P3.json    unweighted interaction and additive risk differences
_results/P4.json    the design x adjustment matrix on matched rows
_results/P5.json    missingness, both estimands
_results/M1.json    M1's own two analyses
_results/paper.json figures quoted from the article, each verified in its PDF
```

`M1_code.qmd` reads P1–P5, adds its own facts, and writes `M1.json` and `paper.json`, so render the five increments before M1. Asked for a result object that is not there, M1 stops rather than quietly standing in for the increment that should have produced it.

No number on the M1 deck was typed by hand. The deck source cites each figure by the name of the fact that produced it — `{{ P4.facts.w_m1_iii }}` — and every reference was resolved against the result objects above when the deck was produced. `M1_replication_deck.md` here is that resolved version, which is why you read values where the source carries names. What transfers to your own M1 is the principle rather than the machinery: a figure cited from the code that computed it cannot drift from it, and a figure retyped onto a slide eventually does.

What that buys is narrow, and worth stating rather than implying. It fixes the numbers, not the sentences around them — numeric drift is prevented structurally, interpretive drift is not — which is why reading the slides is still the check that matters.

The discipline earned its place by catching real defects in these very examples: a deck quoting a design-aware interaction that no increment computed, a P5 submission printing a locked-estimand table its own code never produced, an ALQ130 rule counting NHANES reserved codes 777/999 as drink counts, and a funnel total that no step produced.

Paths are discovered, never hard-coded

Every increment starts `source("../_shared/paths.R"); repro <- find_repro()`, which walks up for `example/repro` or `reproduction`, or honours the `SPPH604_REPRO` environment variable if you set it. No file here names a drive letter, a `/Users/` path or a `/home/` path, so this folder runs from wherever you cloned or unzipped it. That is also the standard your own M1 and M4 repositories are held to: a reviewer has to be able to run your code without editing a path first.

Note on honesty

These model answers deliberately **do not force the paper's exact numbers**. They reproduce the cohort to $N = 6,371$ (paper 6,300) and *explain* the gap, document the alcohol ambiguity rather than tuning it away, and report where the design-aware and imputed estimates change the point estimate. That is what earns the Exceptional band: tracing and defending choices, not matching a target.